



Modelado de bases de datos y construcción de esquemas relacionales

Breve descripción:

Este componente profundiza en el modelado conceptual, lógico y físico de bases de datos, abordando modelos de datos, normalización, construcción del esquema relacional e introducción al lenguaje SQL. Proporciona las bases teóricas y prácticas necesarias para transformar requerimientos del dominio en estructuras relacionales eficientes y coherentes.

Mayo 2026

Tabla de contenido

Introducción	4
1. Fundamentos de modelos de datos	5
1.1. Concepto de modelo de datos	5
1.2. Clasificación de los modelos de datos	6
2. Modelos de datos vigentes.....	9
2.1. Estructura del modelo jerárquico	10
2.2. Concepto y características del modelo de red.....	11
2.3. Modelos actuales (NoSQL, multimodelo).....	12
3. Modelo conceptual de bases de datos	14
3.1. Concepto de modelo conceptual de datos	14
3.2. Modelo entidad–relación (ER).....	15
3.3. Diagramas entidad–relación	18
4. Modelamiento lógico y físico.....	21
4.1. Definición de tablas, campos y atributos.....	21
4.2. Tipos de claves: primaria, foránea y compuesta.....	23
4.3. Diseño de índices.....	24
4.4. Representación física del almacenamiento	26
5. Normalización de bases de datos	30

5.1. Concepto y objetivos de la normalización	30
5.2. Primera, segunda y tercera forma normal.....	32
5.3. Eliminación de redundancias y dependencias.....	35
6. Diseño estructural de la base de datos.....	38
6.1. Organización de la estructura de tablas	38
6.2. Definición de relaciones entre tablas	40
6.3. Integridad referencial	41
7. Introducción al lenguaje de consulta SQL.....	42
7.1. ¿Qué es SQL y para qué sirve?.....	42
7.2. Operaciones fundamentales de SQL.....	44
Síntesis	47
Glosario	48
Referencias bibliográficas	50
Créditos	51

Introducción

El componente formativo Modelado de bases de datos y construcción de esquemas relacionales, desarrolla las bases conceptuales y técnicas necesarias para transformar requerimientos del dominio en estructuras de datos organizadas, coherentes y aplicables en un sistema gestor de bases de datos. Su propósito es fortalecer la comprensión de los modelos vigentes y orientar el diseño estructural de esquemas relacionales alineados con criterios de integridad, consistencia y organización lógica.

A lo largo del componente se aborda el modelado conceptual mediante el enfoque entidad–relación, la transición hacia el nivel lógico con definición de tablas, claves y restricciones, y la representación física del almacenamiento. Asimismo, se profundiza en el proceso de normalización y se introduce el uso básico del lenguaje SQL para la consulta de datos, integrando teoría y práctica dentro de un marco estructurado.

En consecuencia, el componente no solo permite distinguir los fundamentos de los modelos de datos actuales, sino también diseñar bases de datos relacionales que respondan a necesidades reales, articulando comprensión del dominio, rigor técnico y coherencia estructural en cada etapa del proceso.

1. Fundamentos de modelos de datos

Modelar datos no es un ejercicio ornamental; es la actividad que da forma, orden y estructura a la información que una organización necesita para operar. Cada modelo de datos actúa como una “lente conceptual” que permite representar entidades, atributos, relaciones y reglas del negocio con distintos niveles de abstracción.

Sin un modelo, los datos permanecen dispersos, sin un marco formal que permita interpretarlos o convertirlos en información útil. Con un modelo, en cambio, se obtiene una representación coherente, lógica y comunicable del sistema, lo que habilita el diseño de bases de datos relacionales, la integración de procesos y la automatización de operaciones.

Coronel y Morris (2023) enfatizan que estos modelos funcionan como mapas conceptuales que guían el diseño; sin ellos, cualquier esfuerzo de implementación carecería de una base sólida. Asimismo, coronel y Morris (2023) señalan que el modelado permite detectar redundancias, inconsistencias y vacíos antes de que se conviertan en fallas del sistema en producción.

1.1. Concepto de modelo de datos

El modelo de datos es una representación abstracta que describe cómo se organiza, estructura y manipula la información dentro de un sistema. No consiste únicamente en diagramas; es una construcción formal que establece que existe en el dominio, cómo se conecta y bajo qué condiciones opera. A continuación, se sintetizan sus componentes esenciales:

- **Entidades.** Objetos o conceptos del mundo real sobre los que se requiere almacenar información.

- **Atributos.** Propiedades o características que describen a cada entidad.
- **Relaciones.** Vínculos lógicos que conectan entidades entre sí.
- **Reglas de negocio.** Restricciones y condiciones que determinan el comportamiento válido de los datos.

Estos elementos permiten traducir la complejidad de un entorno organizacional en una estructura formal comprensible y operativa. Una característica esencial del modelo de datos es su capacidad de servir como puente entre el mundo real y el mundo computacional. Representa conceptos del negocio mediante símbolos y convenciones que un motor de base de datos puede interpretar. En consecuencia, transforma un dominio complejo en una arquitectura estructurada, consistente y expresable en términos técnicos.

1.2. Clasificación de los modelos de datos

A continuación, se presenta una explicación sistemática sobre la clasificación de los modelos de datos, destacando sus principales enfoques, fundamentos conceptuales y aplicaciones en los sistemas de información contemporáneos.

Video 1. Clasificación de los modelos de datos



[Enlace de reproducción del video](#)

Transcripción del video. Clasificación de los modelos de datos.

En el mundo digital, la información necesita una estructura para poder organizarse y utilizarse.

A medida que la tecnología y las organizaciones crecieron, los modelos de datos evolucionaron para gestionar volúmenes de información cada vez más exigentes.

A continuación, revisaremos su clasificación, sus principales enfoques y aplicaciones.

Los primeros modelos de datos fueron el jerárquico y el de red.

El jerárquico organiza la información como un árbol: cada dato tiene un único superior, mientras que el modelo de red va más allá y permite múltiples conexiones entre los datos.

El modelo relacional cambió las reglas. Tablas conectadas por claves, consultas en SQL y una adopción masiva en empresas, universidades y gobiernos lo convirtieron en el modelo estándar durante décadas.

Más adelante surgió el modelo orientado a objetos, que integra la lógica de programación con el almacenamiento de datos usando clases y herencia.

El crecimiento explosivo de los datos y la complejidad de los entornos distribuidos abrieron paso a una nueva generación de bases de datos: los modelos NoSQL.

Hoy existen los sistemas multimodelo: motores que integran varios enfoques en uno solo para responder a ecosistemas digitales cada vez más complejos y exigentes.

Comprender los modelos de datos permite elegir la mejor solución y construir sistemas de información eficientes y seguros.

Esta clasificación no es estática. Evoluciona conforme surgen nuevas arquitecturas, necesidades de procesamiento masivo y enfoques híbridos de almacenamiento. En consecuencia, la elección de un modelo de datos depende del contexto organizacional, los requisitos de consistencia y escalabilidad, así como del tipo de información que se desea gestionar.

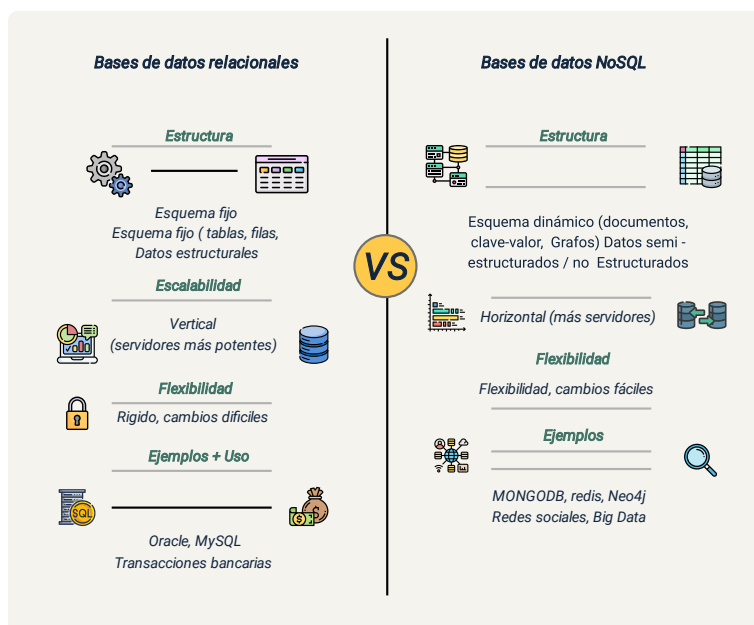
2. Modelos de datos vigentes

A lo largo del tiempo, distintos modelos han predominado según el contexto tecnológico. Comprender su evolución permite analizar por qué el modelo relacional continúa siendo dominante y cómo convive actualmente con alternativas más flexibles.

Este recorrido no debe entenderse como una secuencia lineal en la que un modelo reemplaza completamente a otro. Se trata, más bien, de un diálogo entre diversas formas de organizar la información, cada una pertinente para necesidades específicas como consistencia transaccional, escalabilidad horizontal o manejo de estructuras dinámicas.

Antes de profundizar en sus diferencias, resulta pertinente contrastar las bases de datos relacionales con las soluciones NoSQL, dado que representan los enfoques más influyentes en la actualidad. A partir de esta comparación general, se sintetizan sus principales diferencias conceptuales:

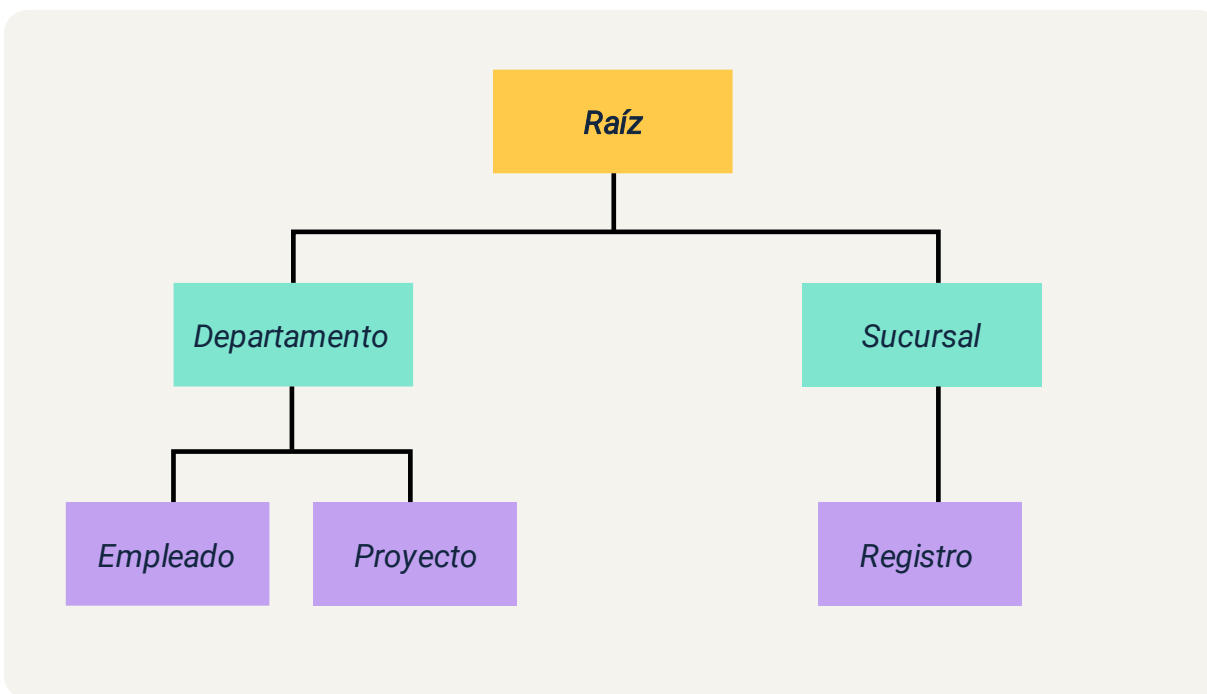
Figura 1. Bases de datos relacionales versus Bases de datos NoSQL



2.1. Estructura del modelo jerárquico

El modelo jerárquico fue uno de los primeros enfoques formales de organización de datos. Se fundamenta en una estructura en forma de árbol, semejante al esquema de carpetas de un sistema operativo. Cada elemento, denominado nodo, mantiene una relación padre–hijo exclusivo: un hijo no puede depender de más de un padre. A continuación, se presenta un esquema representativo de su organización:

Figura 2. Bases de datos relacionales versus Bases de datos NoSQL



Este tipo de estructura se caracteriza por una jerarquía clara y dependencias unidireccionales:

- **Raíz.** Nodo principal del cual dependen todos los demás elementos.
- **Padre.** Nodo que contiene o agrupa otros nodos subordinados.
- **Hijo.** Nodo dependiente de un único padre.
- **Relación.** Vínculo uno a muchos (1:N) entre padre e hijo.

- **Organización.** Estructura rígida en niveles jerárquicos definidos.

Este diseño resulta adecuado cuando la información puede describirse mediante dependencias claramente delimitadas, como ocurre en organigramas institucionales, catálogos de productos o clasificaciones temáticas. Su simplicidad estructural facilita consultas previsibles y rutas de acceso definidas.

No obstante, su rigidez se convirtió en una limitación en contextos donde los datos requieren múltiples relaciones o accesos flexibles. Según Elmasri y Navathe (2007), el modelo jerárquico ofrecía eficiencia operativa, pero sacrificaba flexibilidad y expresividad, lo que condujo a su progresiva disminución en aplicaciones modernas.

2.2. Concepto y características del modelo de red

El siguiente hito en la evolución de los modelos de datos fue el modelo de red, regulado por el estándar CODASYL. A diferencia del modelo jerárquico, este enfoque permite que un registro tenga múltiples padres, lo que habilita un entramado más natural de relaciones entre datos.

Mientras el modelo jerárquico organiza la información en forma de árbol, el modelo de red la concibe como una malla de conexiones. Esta característica posibilitó representar estructuras complejas en sectores como banca, manufactura y logística, donde las relaciones entre entidades suelen ser múltiples y dinámicas. Sus características principales pueden sintetizarse así:

- **Flexibilidad elevada.** Permite modelar relaciones complejas y múltiples entre registros.
- **Acceso navegacional.** El sistema accede a los datos siguiendo enlaces preestablecidos entre registros.

- **Eficiencia de consulta.** El acceso directo a relaciones reduce costos computacionales en entornos estructurados.
- **Complejidad técnica.** Su diseño y mantenimiento requieren mayor conocimiento técnico especializado.

Aunque el modelo de red representó un avance significativo en flexibilidad respecto al jerárquico, su complejidad dificultaba la comprensión por parte de usuarios no técnicos. Por esta razón, fue desplazado en el mercado masivo por el modelo relacional, que introdujo una lógica más intuitiva basada en tablas y consultas declarativas.

2.3. Modelos actuales (NoSQL, multimodelo)

El crecimiento exponencial de los datos, la necesidad de procesamiento en tiempo real y la diversidad de formatos impulsaron el surgimiento de los modelos NoSQL. Estos enfoques priorizan flexibilidad estructural, escalabilidad horizontal y adaptación a entornos distribuidos. A continuación, se presentan sus principales variantes:

- **Bases documentales.** Almacenan datos en documentos JSON o BSON. Son adecuadas cuando los registros no comparten exactamente la misma estructura. La documentación oficial de MongoDB (2026) resalta su aplicabilidad en aplicaciones web, analítica y microservicios.
- **Almacenes clave–valor.** Estructuras simples orientadas a alto rendimiento. Constituyen la base de sistemas de caché como Redis.

- **Bases de datos de grafos.** Representan entidades como nodos y relaciones como aristas. Son útiles en redes sociales, análisis de vínculos, rutas y sistemas de recomendación.
- **Modelos multimodelo.** Motores como *PostgreSQL* o *ArangoDB* integran varios paradigmas en un mismo servidor, permitiendo almacenar simultáneamente documentos, grafos y tablas relacionales.

Antes de analizar su integración en arquitecturas modernas, resulta pertinente considerar su convergencia tecnológica. Los modelos actuales no sustituyen al relacional; lo complementan. Coronel y Morris (2023) sostienen que la elección del modelo depende del tipo de aplicación (transaccional, analítica, distribuida o semiestructurada) y de los requisitos de consistencia, rendimiento y escalabilidad del sistema.

3. Modelo conceptual de bases de datos

Cuando se modelan datos, la abstracción conceptual funciona como un punto intermedio entre la complejidad del mundo real y la formalidad matemática del modelo relacional. Este nivel de diseño, denominado modelo conceptual, ofrece una representación organizada del sistema sin incorporar todavía detalles técnicos como tipos de datos, índices o definiciones físicas.

En los primeros apartados del componente formativo se revisaron los modelos históricos y actuales; ahora se inicia la transición hacia el diseño práctico. En términos de Elmasri y Navathe (2007), el modelo conceptual constituye una “representación semántica” del dominio, es decir, una estructura organizada de la realidad que se desea almacenar.

Por su parte, Coronel y Morris (2023) lo describen como el lenguaje común entre analistas, diseñadores y usuarios, quienes necesitan comprender la organización de los datos sin conocer cómo los gestionará el sistema. En consecuencia, el modelo conceptual se convierte en la columna vertebral del diseño de una base de datos correcta y bien normalizada, pues articula comprensión del dominio y coherencia estructural.

3.1. Concepto de modelo conceptual de datos

A diferencia del modelo lógico (donde ya existen tablas, claves primarias, tipos de datos y restricciones formales), el modelo conceptual se concentra en describir qué es relevante en el sistema y cómo se relacionan sus componentes, omitiendo deliberadamente cualquier consideración tecnológica. Un modelo conceptual sólido responde preguntas fundamentales como las siguientes:

- ¿Qué entidades existen en el dominio?
- ¿Qué características describen a cada entidad?
- ¿Qué vínculos las conectan?
- ¿Qué reglas del negocio condicionan esas relaciones?

Estos interrogantes pueden sintetizarse de la siguiente manera:

- **Entidades.** Identificar los objetos o conceptos relevantes del dominio.
- **Atributos.** Precisar las características que describen cada entidad.
- **Relaciones.** Establecer los vínculos semánticos entre entidades.
- **Reglas de negocio.** Definir restricciones y condiciones que rigen el sistema.

Por ejemplo, en el contexto de una biblioteca, el modelo conceptual reconoce entidades como Libro, Autor, Usuario y Préstamo, y determina cómo se relacionan mediante asociaciones semánticas. En esta fase no se define qué tipo de dato representará el ISBN ni si el autor tendrá un índice por apellido; esas decisiones corresponden al modelamiento lógico posterior.

Este nivel de abstracción permite la participación activa de personas no técnicas, ya que se centra en describir la realidad organizacional y no la tecnología. Entre sus ventajas principales se encuentra la posibilidad de identificar vacíos, supuestos y ambigüedades antes de traducir el diseño al modelo relacional, lo que reduce errores estructurales en etapas posteriores.

3.2. Modelo entidad–relación (ER)

El modelo entidad–relación (ER) es la técnica más extendida para la elaboración de modelos conceptuales. Fue propuesto por Pin-Shan Chen (1976) y continúa vigente

debido a que ofrece una representación clara, comprensible y estandarizada del sistema.

Diversos motores de bases de datos, como Oracle o SQL Server, incorporan herramientas que permiten generar esquemas relacionales a partir de diagramas ER.

Este modelo facilita la transición entre la comprensión del dominio y la formalización lógica, pues organiza los elementos esenciales del sistema en estructuras semánticas claramente definidas.

Para favorecer su apropiación conceptual, puede entenderse el modelo ER como una forma estructurada de relatar la dinámica de un sistema:

- Las entidades representan los “personajes” del dominio.
- Los atributos describen sus características.
- Las relaciones expresan los vínculos o interacciones entre ellos.

Figura 3. Forma simple de ER



Es importante precisar que el modelo entidad–relación admite mayores niveles de detalle que enriquecen su capacidad descriptiva. Entre estos se encuentran las cardinalidades, la participación obligatoria u opcional, los atributos multivaluados, las jerarquías de especialización y generalización, así como las relaciones recursivas, entre otros.

Esta amplitud convierte al modelo ER en una herramienta altamente expresiva, capaz de representar tanto estructuras básicas como configuraciones complejas del dominio organizacional.

Atributos: el comportamiento de las entidades

Los atributos no constituyen un conjunto homogéneo. Su naturaleza varía según la función que desempeñan dentro del sistema y su relación con otras entidades. A continuación, se sintetizan sus principales tipos:

- **Simple.** No puede descomponerse en partes más pequeñas (por ejemplo, edad).
- **Compuesto.** Puede dividirse en subcomponentes con significado propio (por ejemplo, dirección: calle, ciudad, código postal).
- **Identificador (clave).** Distingue de manera única cada instancia de la entidad.
- **Multivaluado.** Permite más de un valor para una misma entidad (por ejemplo, varios números telefónicos).
- **Derivado.** Se obtiene a partir de otros atributos (por ejemplo, edad calculada desde la fecha de nacimiento).

La identificación precisa de estos tipos permite representar con mayor fidelidad la lógica del dominio antes de trasladarla al modelo lógico.

Cardinalidad y participación

La cardinalidad y la participación constituyen la semántica central del modelo ER, pues determinan la naturaleza de las relaciones entre entidades. Estos elementos permiten establecer:

- A. Cuántas instancias de una entidad pueden asociarse con instancias de otra.
- B. Si la relación es obligatoria u opcional.
- C. Si existe dependencia de existencia entre entidades.

Estas reglas pueden resumirse así:

- **Cardinalidad.** Define la correspondencia entre entidades (1:1, 1:N, N:M).
- **Participación.** Indica si todas las instancias deben participar en la relación (total) o si es opcional (parcial).
- **Dependencia de existencia.** Determina si una entidad puede existir sin estar asociada a otra.

Por ejemplo, un Préstamo no puede existir sin un Usuario, mientras que un Usuario puede existir sin realizar préstamos. Esta condición expresa una regla semántica del dominio antes que una restricción tecnológica. El modelo ER permite formalizar estas dependencias conceptuales con claridad, fortaleciendo la coherencia del diseño previo al modelamiento lógico.

3.3. Diagramas entidad–relación

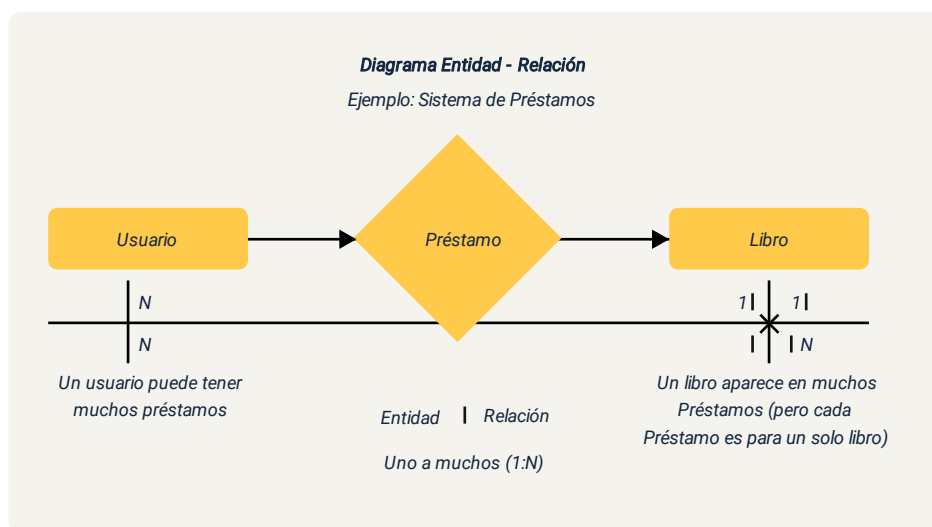
El diagrama entidad–relación (ER) constituye la representación gráfica del modelo conceptual. No es un recurso ornamental, sino una herramienta de análisis y validación. Un diagrama correctamente estructurado permite comprobar la coherencia del sistema sin necesidad de recurrir a descripciones extensas.

Existen diversas notaciones, como Chen, Barker, Crow's Foot y UML. Aunque presentan diferencias formales, todas comparten los mismos componentes esenciales: entidades, atributos, relaciones y cardinalidades. En la práctica profesional, la notación

Crow's Foot es ampliamente utilizada en la documentación técnica asociada a motores como SQL Server y Oracle.

A continuación, se presenta un ejemplo típico con notación Crow's Foot aplicado a un sistema de préstamos:

Figura 4. Ejemplo: sistema de préstamos



En este esquema se identifican dos relaciones fundamentales:

- Un Usuario puede tener muchos Préstamos (relación 1:N).
- Un Libro puede aparecer en muchos Préstamos, pero cada Préstamo se asocia a un único Libro.

Más allá del ejemplo concreto, lo relevante es comprender la función analítica del diagrama ER. Sus principales aportes pueden sintetizarse así:

- **Detección de errores conceptuales.** Permite identificar omisiones, redundancias o inconsistencias antes del diseño lógico.

- **Validación con expertos del dominio.** Facilita la discusión de reglas del negocio con personas no técnicas.
- **Base para el modelo relacional.** Sirve como insumo directo para normalización y construcción de tablas.
- **Claridad estructural.** Reduce ambigüedades en la interpretación del sistema.

Recomendaciones importantes

Aunque no existe un único estilo correcto, es recomendable aplicar criterios de coherencia y claridad:

- Utilizar nombres en singular (Autor en lugar de Autores).
- Representar únicamente los elementos esenciales, evitando sobrecargar el diagrama.
- Confirmar las cardinalidades con expertos del dominio.
- Documentar reglas de negocio que no puedan expresarse mediante la notación gráfica.

El diagrama ER cumple una doble función: es un instrumento técnico y, al mismo tiempo, una herramienta de comunicación. Un diseño conceptual sólido reduce significativamente problemas en las fases posteriores de modelamiento lógico, normalización e implementación.

4. Modelamiento lógico y físico

El modelamiento lógico y físico representa la transformación progresiva del conocimiento conceptual en estructuras preparadas para su implementación en un Sistema Gestor de Bases de Datos (SGBD). El nivel lógico opera en un plano intermedio: no depende aún de un producto comercial específico, pero traduce entidades y relaciones en tablas, claves primarias, claves foráneas y restricciones. El nivel físico, en cambio, se concentra en cómo se almacenan los datos, cómo se distribuyen en páginas o bloques y cómo se optimizan los accesos.

Elmasri y Navathe (2007) señalan que estos niveles buscan “preservar la semántica del dominio mientras se maximiza la eficiencia y consistencia del sistema”. Por su parte, Coronel y Morris (2023) destacan que un modelo físico bien diseñado puede mejorar significativamente el rendimiento sin modificar la lógica del sistema, lo que evidencia la complementariedad de ambos niveles.

Esta relación puede sintetizarse de la siguiente manera:

- **Conceptual.** Describe entidades, atributos y relaciones sin considerar tecnología.
- **Lógico.** Traduce el modelo conceptual en tablas, claves y restricciones formales.
- **Físico.** Define almacenamiento, índices, particionamiento y optimización del rendimiento.

El tránsito entre estos niveles exige coherencia estructural, criterio técnico y comprensión del dominio.

4.1. Definición de tablas, campos y atributos

El primer paso del modelamiento lógico consiste en convertir las entidades del modelo conceptual en tablas, y sus características en columnas o campos. El término

“atributo” se conserva desde el diagrama ER; sin embargo, en el modelo lógico adquiere formalidad: se convierte en un campo con nombre, tipo de dato y restricciones asociadas. Este proceso no implica una simple transferencia mecánica de elementos. Requiere decisiones estratégicas sobre la granularidad de la información. Por ejemplo:

- Un atributo Dirección puede desagregarse en Calle, Ciudad y Código Postal.
- Un atributo Nombre completo puede dividirse en Nombre y Apellido, según las necesidades de consulta y reporte.

Estas decisiones dependen del uso previsto de la base de datos. Si se requieren consultas detalladas o segmentaciones específicas, conviene descomponer atributos compuestos. Si el objetivo es simplicidad operativa, puede mantenerse una estructura más compacta. A continuación, se presenta un ejemplo de transformación del nivel conceptual al lógico:

Entidad conceptual: **USUARIO**

Atributos conceptuales: Nombre, Documento, Correo, Teléfono

Antes de presentar la tabla, es pertinente aclarar que los tipos de datos y las restricciones se definen con base en criterios de integridad, unicidad y consistencia estructural.

Tabla 1. Estructura lógica de la entidad Usuario

Campo	Tipo sugerido	Restricción
id_usuario	INT	PK
documento	VARCHAR	UNIQUE
nombre	VARCHAR	NOT NULL

Campo	Tipo sugerido	Restricción
correo	VARCHAR	NOT NULL
telefono	VARCHAR	NULL

Esta estructura garantiza identificación única, evita duplicidad en el documento y establece obligatoriedad en campos esenciales. No existe una única configuración correcta. Lo fundamental es que la tabla represente fielmente la semántica del dominio, preserve la coherencia con el modelo conceptual y facilite el uso operativo futuro, tanto en consultas como en generación de reportes y mantenimiento del sistema.

4.2. Tipos de claves: primaria, foránea y compuesta

Las claves constituyen el mecanismo formal que garantiza la identidad, coherencia e integridad dentro del modelo relacional. No son simples valores almacenados en una tabla; funcionan como anclas estructurales que previenen inconsistencias y sostienen las relaciones entre datos. A continuación, se describen los principales tipos de claves y su función dentro del modelamiento lógico:

- **Clave primaria (PK).** Identifica cada registro de manera inequívoca. Se recomienda que sea estable, que no cambie con el tiempo; simple, preferiblemente conformada por un solo campo; y sin significado operacional mutable, evitando datos susceptibles de modificación, como números de documento sujetos a actualización normativa.
- **Clave foránea (FK).** Representa el vínculo formal entre tablas y se deriva de las relaciones definidas en el modelo conceptual. Su función es asegurar que un valor registrado en una tabla exista previamente en otra, preservando la

integridad referencial. Por ejemplo, si una tabla Prestamo contiene el campo `id_usuario`, este debe corresponder a un registro existente en la tabla Usuario.

- **Clave compuesta.** Se presenta cuando la unicidad de un registro depende de la combinación de varios campos. En una tabla DetalleFactura, la combinación (`id_factura`, `id_producto`) puede definir la identidad del registro, ya que un producto puede aparecer en distintas facturas y una factura puede incluir varios productos. Aunque resultan útiles cuando reflejan una relación natural entre entidades, su uso excesivo puede complejizar el diseño, afectar el rendimiento y dificultar la administración del sistema. Por ello, se recomienda emplearlas únicamente cuando representen de forma genuina la estructura semántica del dominio.

4.3. Diseño de índices

Los índices son estructuras adicionales que permiten acelerar la recuperación de datos, funcionando como rutas alternativas de acceso a la información. Aunque no forman parte del modelo conceptual, son elementos esenciales en el nivel físico del diseño de bases de datos.

Para dimensionar su impacto, conviene considerar lo siguiente: una búsqueda sin índice puede implicar recorrer todos los registros de una tabla. Con un índice adecuado, el motor identifica directamente la ubicación del registro requerido, reduciendo significativamente el tiempo de respuesta.

Este contraste puede comprenderse mediante la siguiente analogía:

- **Sin índice.** Buscar un libro en un estante desordenado revisando ejemplar por ejemplar.

- **Con índice.** Consultar un catálogo organizado y dirigirse directamente al ejemplar indicado.

A continuación, se presentan los tipos de índices más comunes en sistemas relacionales:

- **Índice simple.** Basado en un único campo. Adecuado para búsquedas frecuentes por un atributo específico.
- **Índice compuesto.** Construido sobre dos o más campos. Útil cuando las consultas incluyen múltiples filtros combinados.
- **Índice único.** Garantiza que no existan valores duplicados en el campo indexado. Refuerza la integridad de los datos.
- **Índice de texto completo.** Diseñado para búsquedas en campos extensos o documentos, especialmente en consultas por palabras clave.

El diseño de índices exige equilibrio técnico. Un número excesivo puede afectar negativamente el rendimiento en operaciones de inserción, actualización o eliminación, ya que el SGBD debe mantener cada índice sincronizado con los datos almacenados.

En este sentido, Coronel y Morris (2023) recomiendan crear índices únicamente sobre campos que se utilicen con frecuencia en filtros, ordenamientos o uniones (JOIN). De esta manera, se optimiza el rendimiento sin comprometer la eficiencia global del sistema.

4.4. Representación física del almacenamiento

Una vez definidos los elementos del modelo lógico, surge una pregunta fundamental: ¿cómo se almacenan realmente los datos en un Sistema Gestor de Bases de Datos (SGBD)? En el nivel físico, los datos no permanecen como tablas abstractas, sino que se organizan en estructuras internas optimizadas para almacenamiento y recuperación eficiente. Aunque cada motor implementa mecanismos propios, los principios generales son similares: dividir, agrupar y administrar la información de forma que se garantice rendimiento, seguridad e integridad. Los componentes físicos más relevantes pueden sintetizarse así:

- **Páginas o bloques.** Unidades mínimas de almacenamiento donde se guardan los registros. Su tamaño suele ser fijo (por ejemplo, 8 KB o 16 KB).
- **Archivos de datos.** Contenedores físicos que agrupan páginas y son gestionados internamente por el motor del SGBD.
- **Índices físicos.** Estructuras jerárquicas (como árboles B+) que optimizan la localización de registros.
- **Buffer pools.** Áreas de memoria que mantienen datos activos para reducir accesos directos al disco.

Diversos motores como Oracle, PostgreSQL y SQL Server describen en su documentación oficial que:

- Las tablas se almacenan en páginas o bloques de tamaño fijo.
- Los índices emplean estructuras jerárquicas para acelerar la búsqueda.
- La información más utilizada se mantiene en memoria intermedia (buffer pool).

- Los registros se agrupan en archivos físicos administrados por el motor.

En este nivel comienzan a influir factores como la fragmentación, la paginación, la ubicación física de los datos y la compresión, todos determinantes en el desempeño del sistema, especialmente cuando se gestionan miles o millones de registros. Una analogía permite comprender la diferencia entre niveles:

Si el modelo lógico define qué habitaciones tiene una casa y cómo se relacionan entre sí, el modelo físico determina cómo se distribuyen los ladrillos, qué tipo de cimiento soporta la estructura y cómo se organizan las conexiones internas que garantizan su funcionamiento eficiente.

En consecuencia, el diseño físico no altera la semántica del dominio, pero impacta directamente en la eficiencia operativa, la escalabilidad y la estabilidad del sistema. La representación física del almacenamiento permite comprender cómo el SGBD organiza internamente páginas, bloques, archivos de datos, memoria intermedia (buffer pool) e índices para garantizar acceso eficiente y consistencia operativa.

La representación física del almacenamiento define cómo se organizan los datos para mejorar el rendimiento del sistema:

- **Representación física del almacenamiento de bases de datos.** De la lógica a los ladrillos digitales.
- **Cómo se almacenan los datos.** Los SGBD utilizan páginas, bloques, páginas de buffer y buffers de memoria.
- **Buffer.** Espacio temporal en memoria para gestionar la lectura y escritura de datos.
- **Procesos asociados.** Fragmentación, paginación, ubicación física y compresión.

- **Analogía.** Si el modelo lógico define las habitaciones, el modelo físico define los ladrillos y los cimientos.
- **Optimización del modelo físico.** Las decisiones se toman con base en patrones de consulta, patrones de acceso, patrones de negocio, reglas de negocio y requisitos no funcionales.
- **Ajustes posibles.** Ajuste de columnas, tipos de índices, tipos específicos, particiones grandes y ubicación en discos (paralelismo).
- **Clave para el rendimiento.** Diferencias de tiempo entre milisegundos y segundos. Referencias: AWS (2025) y SQL Server (2026).
- **Impacto.** Influye de forma directa en la experiencia del usuario y en la operación del sistema.

Optimización del modelo físico

Optimizar no significa “hacer más rápido” sin criterio técnico. Implica tomar decisiones estratégicas sustentadas en:

- Patrones de consulta frecuentes.
- Volumen esperado de datos.
- Reglas del negocio que condicionan integridad y acceso.
- Requerimientos no funcionales, como disponibilidad, escalabilidad y tiempos de respuesta.

Las principales acciones de optimización pueden sintetizarse así:

- **Ajuste de longitud de columnas.** Reduce consumo de almacenamiento y mejora eficiencia en lectura.

- **Selección adecuada de tipos de datos.** Disminuye uso innecesario de espacio y mejora rendimiento.
- **Creación de índices específicos.** Optimiza consultas complejas y operaciones de filtrado.
- **Particionamiento de tablas grandes.** Facilita administración y mejora tiempos de acceso en grandes volúmenes.
- **Distribución de archivos en distintos discos.** Incrementa paralelismo y rendimiento en operaciones intensivas.

Amazon Relational Database Service (2026) y Microsoft (2026) explican que la optimización del almacenamiento puede mejorar el rendimiento incluso sin modificar una sola línea de SQL. Esto demuestra que el diseño físico tiene un impacto directo en el comportamiento del sistema. En bases de datos críticas, estas decisiones pueden determinar si una consulta tarda milisegundos o varios segundos, lo que incide directamente en la experiencia del usuario y en la continuidad operativa de la organización.

5. Normalización de bases de datos

La normalización constituye uno de los procesos centrales en el diseño de bases de datos relacionales. Aunque suele asociarse con reglas y formas normales específicas, su propósito es más amplio: eliminar redundancias innecesarias, preservar la integridad lógica del dominio y facilitar consultas consistentes y eficientes.

Elmasri y Navathe (2007) definen la normalización como una “técnica para analizar relaciones y asegurar que estén estructuradas de modo que minimicen las anomalías de actualización”. Estas anomalías incluyen problemas que deterioran la confiabilidad del sistema, tales como duplicación de datos, inconsistencias entre registros o dificultades para insertar y eliminar información sin efectos colaterales.

En la práctica, la normalización no es una etapa aislada. Forma parte de un proceso iterativo en el que se revisa el modelo conceptual, se evalúan atributos, se ajustan dependencias funcionales y se reorganizan relaciones. Incluso bases de datos ya implementadas pueden requerir procesos de normalización cuando aparecen inconsistencias históricas o limitaciones estructurales.

5.1. Concepto y objetivos de la normalización

El principio fundamental de la normalización es directo: cada dato debe almacenarse exactamente donde corresponde, sin duplicaciones innecesarias ni dependencias incorrectas. Sus objetivos pueden organizarse en cuatro ejes principales:

- **Eliminar redundancias.** Evita repetición innecesaria de información que incrementa riesgo de inconsistencia.
- **Prevenir anomalías.** Reduce problemas de inserción, eliminación y actualización.

- **Asegurar dependencias lógicas.** Garantiza que los atributos dependan del identificador correcto y no de campos circunstanciales.
- **Facilitar mantenimiento e integridad referencial.** Simplifica reglas de negocio y mejora la administración a largo plazo.

Estos objetivos se concretan de la siguiente manera:

- Cuando una misma información se repite en múltiples lugares, aumentan las probabilidades de error: valores divergentes, actualizaciones incompletas o datos obsoletos.
- Una tabla mal estructurada puede impedir insertar registros sin información innecesaria, eliminar filas sin perder datos relevantes o actualizar campos sin generar inconsistencias.
- Las dependencias funcionales deben vincularse al identificador correcto; de lo contrario, se debilita la coherencia estructural del sistema.
- Un diseño normalizado reduce complejidad técnica y mejora la sostenibilidad del modelo en el tiempo.

Coronel y Morris (2023) advierten que la normalización no debe aplicarse de manera mecánica o indiscriminada. En contextos analíticos o de grandes volúmenes de datos, puede admitirse cierta desnormalización controlada para optimizar consultas. Sin embargo, en sistemas transaccionales, la normalización continúa siendo un estándar indispensable para garantizar integridad y estabilidad.

5.2. Primera, segunda y tercera forma normal

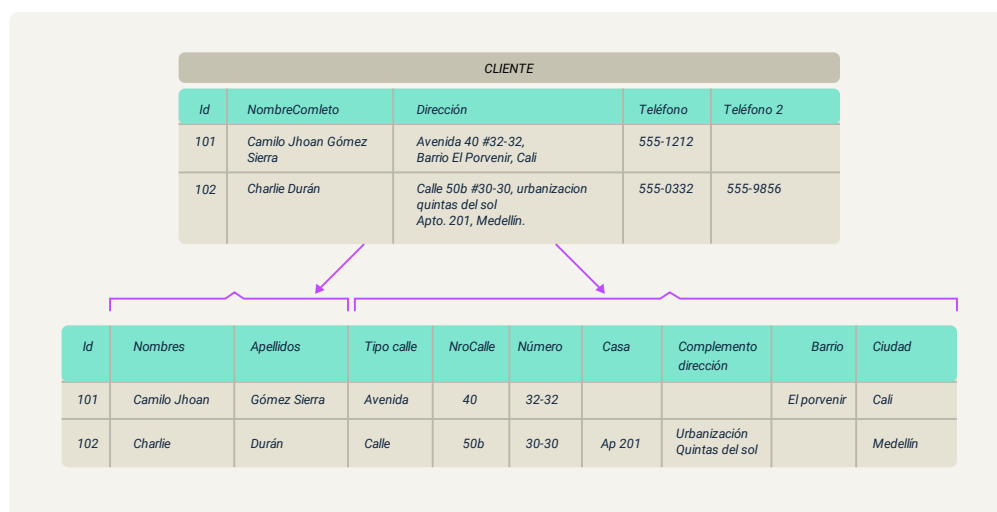
En esta etapa, el diseño comienza a adquirir forma estructural concreta. Las tres primeras formas normales constituyen el núcleo de la normalización clásica y permiten eliminar redundancias progresivamente, asegurando coherencia lógica entre atributos.

Primera Forma Normal (1FN)

Una relación se encuentra en Primera Forma Normal (1FN) cuando todos sus atributos contienen valores atómicos. Esto significa que cada campo debe almacenar un único valor indivisible; no pueden existir listas, conjuntos ni valores compuestos dentro de una misma celda. Este principio responde al criterio básico de atomicidad: cada intersección entre fila y columna debe contener un solo dato.

Por ejemplo, un campo denominado Nombre completo puede requerir separación en Nombres y Apellidos, dependiendo de las necesidades de consulta. De manera similar, un atributo Dirección suele dividirse al menos en Calle, Ciudad y otros componentes relevantes.

Figura 5. Ejemplo de aplicación de la 1FN



En síntesis, la 1FN establece el punto de partida de la normalización: garantizar que cada atributo contenga un único valor elemental, condición necesaria para avanzar hacia formas normales superiores.

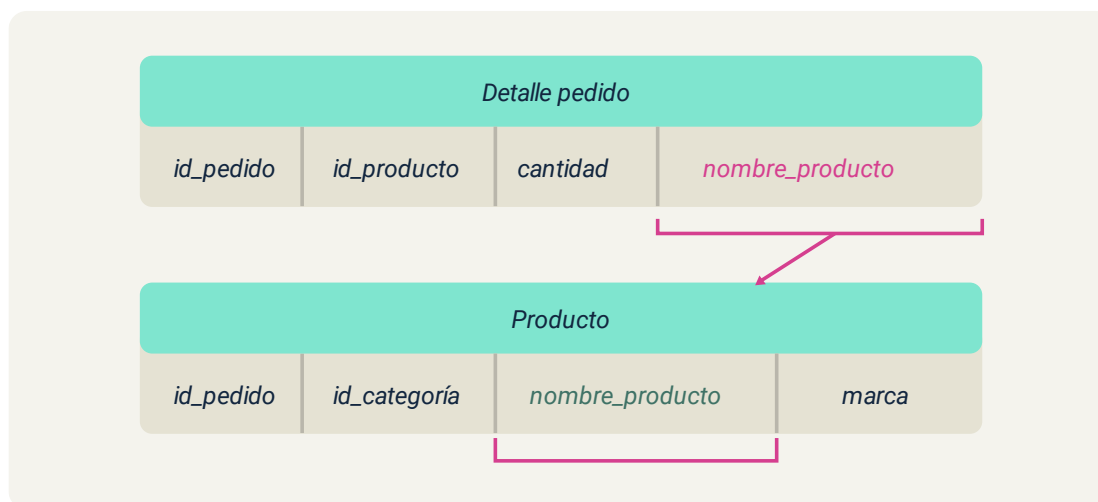
Segunda Forma Normal (2FN)

La Segunda Forma Normal (2FN) aplica específicamente a relaciones cuya clave primaria es compuesta, es decir, está formada por dos o más atributos. Una tabla se encuentra en 2FN cuando todos los atributos no clave dependen de la totalidad de la clave primaria y no solo de una parte de ella. Cuando un atributo depende únicamente de uno de los componentes de la clave compuesta, se presenta una dependencia parcial, lo que constituye una anomalía estructural. Este problema es frecuente en tablas de detalle. Considérese el siguiente caso conceptual:

DetallePedido(id_pedido, id_producto, cantidad, nombre_producto)

Si en esta tabla se incluye el atributo **nombre_producto**, surge una inconsistencia lógica. El nombre del producto no depende de la combinación (id_pedido, id_producto), sino exclusivamente de **id_producto**. Por tanto, ese atributo pertenece a la entidad **Producto** y no debería almacenarse en la tabla de detalle. Antes de la representación gráfica, es importante precisar que la 2FN elimina dependencias parciales, fortaleciendo la coherencia entre identificadores y atributos descriptivos.

Figura 6. Ejemplo de aplicación de la 2FN



En términos prácticos, la 2FN actúa como un filtro que impide que atributos propios de entidades externas se integren indebidamente en tablas cuya clave compuesta no los determina completamente. De esta manera, se reduce redundancia y se refuerza la consistencia estructural del modelo relacional.

Tercera Forma Normal (3FN)

Una relación se encuentra en Tercera Forma Normal (3FN) cuando cumple dos condiciones:

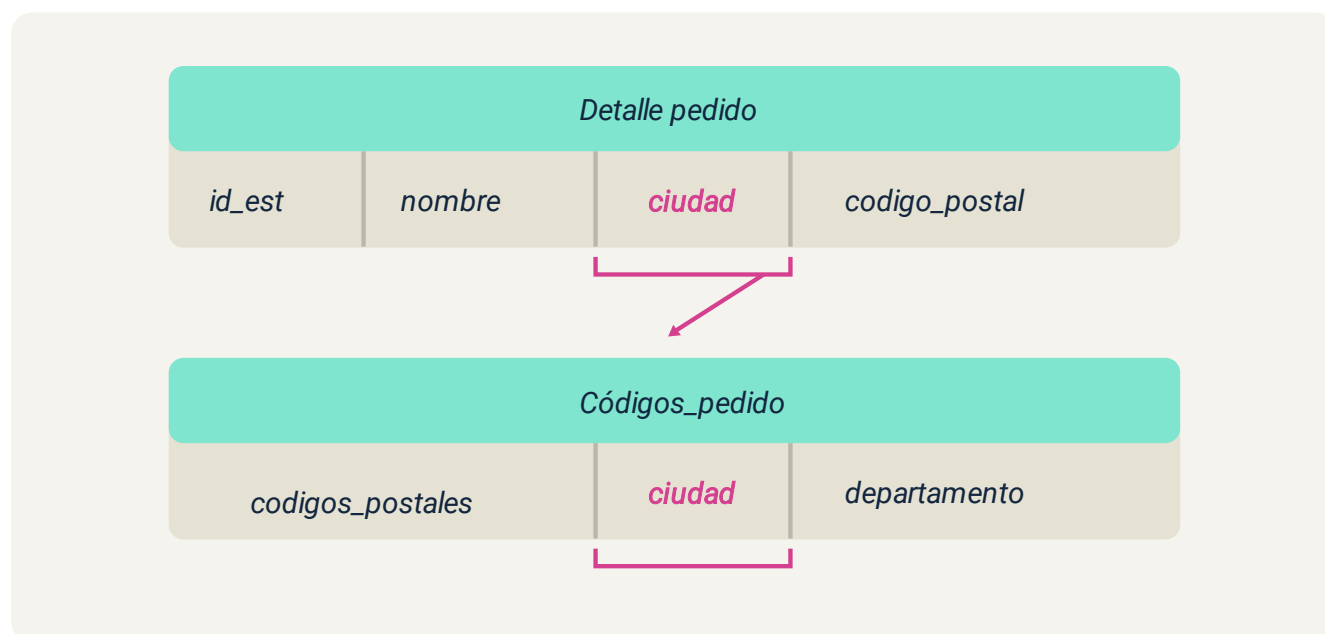
- Está previamente en Segunda Forma Normal (2FN).
- No existen dependencias transitivas entre atributos no clave.

En términos simples, un atributo no clave no debe depender de otro atributo que tampoco sea clave. Toda dependencia debe vincularse directamente con la clave primaria. Considérese el siguiente ejemplo clásico:

Estudiante(id_est, nombre, ciudad, codigo_postal)

Si el atributo ciudad depende de codigo_postal, y este último no es la clave primaria sino un atributo adicional, se configura una dependencia transitiva. En este caso, ciudad depende indirectamente de id_est a través de codigo_postal. Antes de la representación gráfica, es importante aclarar que la 3FN elimina este tipo de dependencias indirectas para fortalecer la coherencia estructural del modelo.

Figura 7. Ejemplo de aplicación de la 3FN



Al alcanzar la 3FN, la estructura resultante suele ser suficientemente robusta para la mayoría de las aplicaciones transaccionales, ya que reduce redundancia, previene anomalías y mantiene coherencia lógica entre los datos.

5.3. Eliminación de redundancias y dependencias

La eliminación de redundancias constituye tanto un resultado como un indicador de una normalización adecuada. No obstante, es necesario precisar que no toda redundancia implica un error estructural. En sistemas orientados a analítica, data

warehouses o motores NoSQL, la redundancia puede utilizarse estratégicamente para mejorar el rendimiento y simplificar consultas complejas.

Sin embargo, en el modelo relacional tradicional (especialmente en entornos transaccionales) la redundancia innecesaria suele generar consecuencias negativas que afectan la integridad y sostenibilidad del sistema. Estas consecuencias pueden sintetizarse así:

- **Inconsistencias lógicas.** Diferencias entre registros que deberían contener la misma información.
- **Duplicidad silenciosa.** Repetición de datos sin mecanismos claros de control.
- **Dificultad de mantenimiento.** Incremento del esfuerzo para modificar o auditar información.
- **Pérdida de integridad.** Riesgo de que reglas del negocio no se cumplan uniformemente.

La normalización permite depurar estas redundancias mediante la reorganización estructural de las tablas. Una técnica clave en este proceso consiste en identificar dependencias funcionales, es decir, reglas que determinan cómo un conjunto de atributos define a otros dentro de una relación.

Una dependencia funcional se expresa de la siguiente forma:

- (NumeroDocumento) → (Nombre, Apellido, FechaNacimiento)
- Esto significa que el NúmeroDocumento determina de manera única el Nombre, el Apellido y la Fecha de Nacimiento.

Si una dependencia funcional no está alineada con la clave primaria de la tabla, o si revela que ciertos atributos dependen de otro campo que no es clave, es probable que la relación requiera descomposición en entidades separadas. Esta reorganización fortalece la coherencia lógica del modelo y reduce anomalías estructurales.

6. Diseño estructural de la base de datos

Una vez que el modelo conceptual ha sido transformado en tablas normalizadas, y después de definir claves, dependencias y relaciones, es necesario examinar la arquitectura completa del esquema relacional. En esta etapa se analiza cómo se articulan las tablas, cómo se organizan dentro del esquema y cómo se garantiza la coherencia en cada operación del sistema.

El diseño estructural no constituye únicamente un plano técnico; también representa el funcionamiento organizacional. Las decisiones adoptadas en este punto impactan directamente en la eficiencia operativa, la claridad de las consultas y la capacidad de adaptación ante cambios del negocio.

6.1. Organización de la estructura de tablas

La organización de las tablas no responde a una fórmula rígida. Depende de la naturaleza del dominio, del volumen esperado de datos y del tipo de operaciones predominantes (transaccionales, analíticas o híbridas). En términos generales, una estructura bien organizada incorpora los siguientes componentes:

- **Tablas principales.** Representan entidades centrales del dominio (Cliente, Producto, Pedido).
- **Tablas puente.** Resuelven relaciones N:M mediante claves foráneas hacia las entidades relacionadas.
- **Tablas auxiliares.** Contienen catálogos, parámetros o listas controladas (Estados, Tipos, Categorías).
- **Tablas de auditoría.** Registran eventos, cambios o acciones realizadas sobre los datos.

Un ejemplo muy breve, sin entrar en casos reales complejos:

USUARIO

|— id_usuario

|— nombre

└— correo

LIBRO

|— id_libro

|— titulo

└— categoría

PRESTAMO

|— id_prestamo

|— id_usuario (FK)

|— id_libro (FK)

|— fecha_prestamo

└— fecha_devolucion

CATEGORIA

|— id_categoria

└— descripcion

La estructura puede crecer y volverse considerablemente más compleja, pero el principio es el mismo: cada tabla representa un concepto y está ubicada en un nivel coherente dentro del esquema.

6.2. Definición de relaciones entre tablas

Las relaciones constituyen el eje estructural del modelo relacional. Su propósito no se limita a conectar tablas; buscan establecer vínculos con reglas claras que reflejen la lógica del dominio. Las relaciones pueden clasificarse en tres tipos principales:

- **Uno a uno (1:1).** Poco frecuentes en modelos operativos. Se utilizan en fragmentaciones lógicas o cuando se separan datos sensibles en tablas distintas.
- **Uno a muchos (1:N).** La más común en sistemas transaccionales. Una instancia de una entidad puede relacionarse con varias de otra, pero no a la inversa.
- **Muchos a muchos (N:M).** No se implementa directamente en el modelo relacional; requiere una tabla intermedia o tabla puente que contenga las claves foráneas de ambas entidades.

Por ejemplo, un Usuario puede tener varios Préstamos, pero cada préstamo pertenece a un único usuario (relación 1:N). En cambio, la relación entre Libro y Autor suele ser N:M, lo que exige una tabla intermedia como LIBRO_AUTOR.

El diseño adecuado de las relaciones facilita consultas eficientes, evita duplicidad innecesaria y permite representar con precisión reglas del negocio que pueden evolucionar con el tiempo. Un error frecuente consiste en considerar las relaciones como simples conexiones gráficas. En realidad, representan estructuras semánticas

profundas: pueden expresar propiedad, dependencia, temporalidad o composición, según el contexto del dominio.

6.3. Integridad referencial

La integridad referencial garantiza que el conjunto de tablas funcione como un sistema coherente. No solo implica que una clave foránea (FK) deba existir previamente como clave primaria (PK) en otra tabla; también define el comportamiento del sistema cuando los datos cambian. Los SGBD modernos permiten establecer políticas específicas para estos casos:

- **CASCADE.** Propaga automáticamente la eliminación o actualización a los registros relacionados.
- **SET NULL.** Asigna valores nulos a las claves foráneas dependientes.
- **RESTRICT / NO ACTION.** Impide la operación si existen registros relacionados activos.

Estas reglas son fundamentales para preservar la coherencia estructural. Un sistema sin integridad referencial puede presentar referencias a registros inexistentes, datos incompletos o inconsistencias que deterioran progresivamente la calidad de la información. Motores empresariales como Oracle Database, SQL server y PostgreSQL documentan ampliamente la importancia de definir políticas explícitas de actualización y eliminación, ya que estas previenen fallos silenciosos y protegen la semántica del modelo diseñado.

7. Introducción al lenguaje de consulta SQL

SQL (Structured Query Language) surgió en la década de 1970 como un lenguaje declarativo diseñado para consultar y manipular bases de datos relacionales sin requerir programación algorítmica tradicional. Su principal fortaleza radica en que permite expresar qué se desea obtener, mientras el sistema gestor decide cómo ejecutar la operación.

Esta característica lo convierte en un lenguaje accesible para analistas, consultores, administradores y desarrolladores. Además, constituye un estándar implementado por prácticamente todos los Sistemas Gestores de Bases de Datos Relacionales (SGBDR), aunque cada proveedor puede incorporar extensiones propias.

SQL no opera de manera aislada: depende directamente de la estructura diseñada previamente. Por ello, comprender el modelo relacional es condición indispensable para formular consultas coherentes y eficientes.

7.1. ¿Qué es SQL y para qué sirve?

A continuación, se presenta un pódcast orientado a comprender los fundamentos del lenguaje SQL y su aplicación en la consulta de información dentro de bases de datos relacionales.

Transcripción pódcast. ¿Qué es SQL y para qué sirve?
Hombre: Bienvenidos. Hoy vamos a hablar del lenguaje que mueve los datos en el mundo digital: el SQL. Aunque suene a código complejo, en realidad es la herramienta que nos permite hablar con las bases de datos para encontrar lo que necesitamos.

Transcripción pódcast. ¿Qué es SQL y para qué sirve?

Mujer: Para entenderlo fácil, imaginen que la base de datos es la memoria de una red social con millones de fotos y videos. El SQL es el motor que trabaja detrás de la lupa; cuando usted busca una etiqueta, ese lenguaje es el que revuelca entre millones de publicaciones para mostrarle el video exacto en segundos.

Hombre: El SQL funciona exactamente como el GPS de la información. Uno simplemente marca el destino y el sistema se encarga de encontrar la ruta más corta entre millones de datos, directo al punto exacto y sin perderse en el mapa de las tablas.

Mujer: Pero ojo, que el SQL no solo sirve para “mirar”. También nos permite meter datos nuevos, arreglar los que quedaron mal o borrar lo que ya no sirve, manteniendo la información siempre al día.

Hombre: Las funciones de SQL se reparten según la tarea. La más famosa es la consulta, que es pedirle al sistema que nos traiga datos específicos, incluso si están guardados en tablas diferentes al mismo tiempo.

Mujer: También es el que manda en la casa. Permite armar la estructura de las tablas desde cero y, algo muy importante, controla quién tiene permiso para ver o tocar la información. No cualquiera puede entrar a mover los datos.

Hombre: Hay un punto muy importante que siempre se debe tener en cuenta: para mandar bien en SQL, primero hay que entender el modelo relacional. No se trata de escribir comandos por escribir; hay que saber cómo se conectan las tablas entre sí por medio de sus atributos.

Mujer: Es como aprender a leer un mapa antes de arrancar a conducir. Primero hay que entender dónde estamos parados, y ahí sí, lanzarse al lenguaje de consulta.

Transcripción pódcast. ¿Qué es SQL y para qué sirve?

Hombre: Dominar el SQL no es solo aprender un código; es aprender el lenguaje que sostiene el mundo moderno. El reto es transformar esos datos en decisiones efectivas.

Mujer: Llegamos al final de nuestro programa. Gracias por acompañarnos. ¡Nos escuchamos en el próximo episodio!

7.2. Operaciones fundamentales de SQL

A continuación, se presentan las operaciones esenciales del lenguaje SQL orientadas a la consulta de datos dentro del modelo relacional. Los siguientes ejemplos no constituyen un tutorial exhaustivo. Su propósito es ilustrar cómo funcionan las consultas dentro de una base de datos estructurada.

SELECT: recuperar datos

La forma más básica de extraer información de una tabla:

SELECT*FROM usuario;

La expresión * recupera todas las columnas. No es recomendable en sistemas grandes, pero es útil para exploración.

SELECT con columnas específicas

La forma más básica de extraer información de una tabla:

SELECT nombre, correo

FROM usuario;

WHERE: aplicar condiciones

Permite filtrar registros según criterios:

SELECT titulo, categoría

FROM libro

WHERE categoría = tecnología;

Operadores (=, <, >, LIKE, BETWEEN)

La forma más básica de extraer información de una tabla:

SELECT titulo

FROM libro

WHERE titulo like %datos%;

SELECT nombre

FROM usuario

WHERE id_usuario BETWEEN 10 AND 20;

ORDER BY: ordenar resultados

Ordenar registros en forma ascendente o descendente:

SELECT titulo, categoria

FROM libro

ORDER BY titulo ASC;

LIMIT: restringir resultados

Útil para análisis preliminares o listados rápidos:

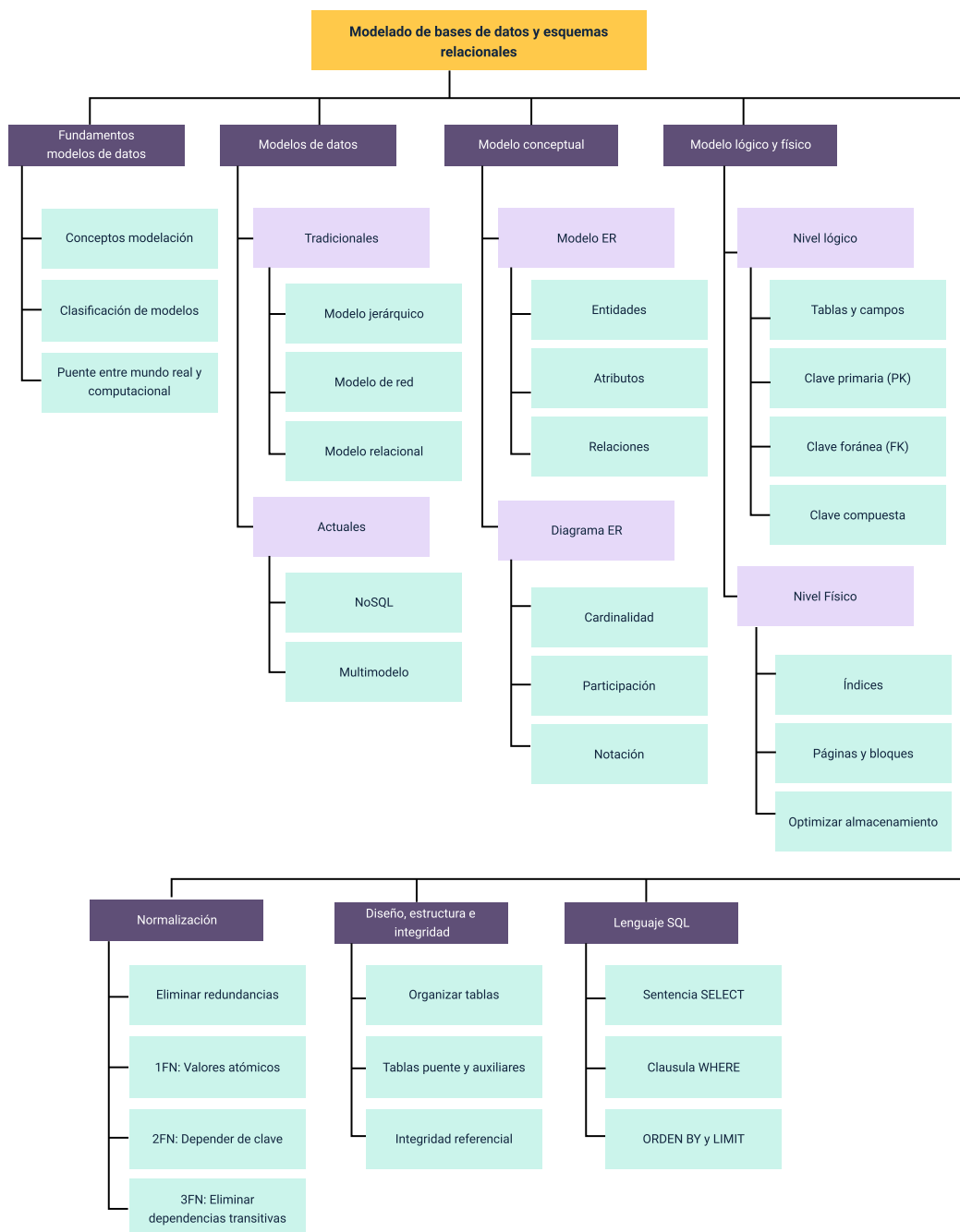
SELECT *

FROM prestamo

Algunos motores utilizan TOP (SQL Server) o FETCH FIRST (Oracle), pero el principio es universal.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Glosario

Atributo: característica que describe a una entidad; en el modelo lógico se implementa como una columna de tabla.

Clave foránea (FK): campo que referencia una clave primaria de otra tabla, garantizando integridad referencial.

Clave primaria (PK): atributo o conjunto de atributos que identifica de manera única un registro dentro de una tabla.

Data warehouses: bodega electrónica donde una empresa u organización almacena grandes volúmenes de información, generalmente provenientes de diversas fuentes y bases de datos.

Dominio: conjunto de valores o atributos que comparten un significado o propósito común dentro de la gestión de datos; representa el límite del sistema.

Entidad: objeto o concepto relevante para el dominio, representado en el modelo conceptual y posteriormente convertido en tabla.

Grafos: representaciones que modelan metadatos complejos o conceptos de dominio en un formato estandarizado para aportar semántica al procesamiento del lenguaje natural.

Índice: estructura física auxiliar que acelera la búsqueda de datos en una tabla.

JSONB: tipo de datos de PostgreSQL similar a JSON que se utiliza para almacenar datos semiestructurados.

N:M: notación utilizada para denominar una relación de muchos a muchos; comúnmente asociada con 1:N, que indica una relación uno a muchos.

Paralelismo: contexto en el que se emplean varios discos duros para mejorar el rendimiento o la disponibilidad del sistema.

Relación: vínculo entre dos entidades, expresado mediante cardinalidades y, en el modelo relacional, mediante claves foráneas.

Validación: comprobación y limpieza de los datos para evitar que ingresen incorrectos, incompletos o irrelevantes a la base de datos, garantizando su fiabilidad.

Referencias bibliográficas

Amazon Web Services. (2026). *Introducción a Amazon Relational Database Service (Amazon RDS)*.

https://docs.aws.amazon.com/es-es/AmazonRDS/latest/UserGuide/CHAP_GettingStarted.html

Coronel, C., y Morris, S. (2023). *Database systems: Design, implementation, and management*. Cengage Learning.

<https://faculty.cengage.com/works/9780357673034>

Elmasri, R., y Navathe, S. B. (2007). *Fundamentals of database systems* (7th ed.). Pearson.

Microsoft. (2026). *Documentación técnica de SQL Server*.

<https://learn.microsoft.com/es-es/sql/sql-server/?view=sql-server-ver17>

MongoDB. (2026). *Documentación de MongoDB*.

<https://www.mongodb.com/es/docs/>

Oracle Corporation. (2025). *Database concepts*.

<https://docs.oracle.com/en/database/oracle/oracle-database/19/cncpt/index.html>

Chen, P. P.-S. (1976). The entity-relationship model—Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36.

<https://doi.org/10.1145/320434.320440>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Líder del Ecosistema	Centro Agroturístico - Regional Santander
Olga Constanza Bermúdez Jaimes	Responsable de línea de producción Huila	Dirección General
Jaime Hernán Tejada Llano	Experto temático	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Paola Alexandra Moya	Evaluadora instruccional	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Carlos Julian Ramírez Benitez	Diseñador de contenidos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Henry Álvarez Astudillo	Desarrollador full stack	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Alejandro Delgado Acosta	Intérprete lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Cristhian Giovanni Gordillo Segura	Intérprete Lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Juan Pablo Rojas Polania	Animador y productor audiovisual	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Carlos Eduardo Garavito Parada	Animador y productor audiovisual	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Maria Carolina Tamayo Lopez	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
German Acosta Ramos	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Javier Ricardo Ortiz Puentes	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Aixa Natalia Sendoya Fernández	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Daniel Ricardo Mutis Gómez	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila

Nombre	Cargo	Centro de Formación y Regional
Anyerson Wilfredo Pizo Ossa	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila